Assignment 2: Network on Chip

Released: *Sep 28th 2026* • Due: *Oct 10th 2026*, 23:59 IST • Weightage: *10*% of the course grade

1 The Network on Chip

1.1 Topology and addressing

The network is a 4×4 mesh of sixteen routers, each with one PE attached. A router and its PE share an address, the coordinate pair (x, y): x is the column, counted from 0 at the left, and y is the row, counted from 0 at the top. The top-left router is therefore $(0, 0)$ and the bottom-right router is $(3, 3)$. Each coordinate is a 2-bit number.

Directions are named as on a map, with north at the top. For the router at (x, y):

- the **north** port connects to $(x,\ y - 1)$ and the **south** port to $(x,\ y + 1)$;
- the **west** port connects to $(x - 1,\ y)$ and the **east** port to $(x + 1,\ y)$.

A port that would point off the grid — the north ports of the top row, the west ports of the left column, and so on — is unused. Nothing will ever arrive on it, and your router must never send anything out of it.

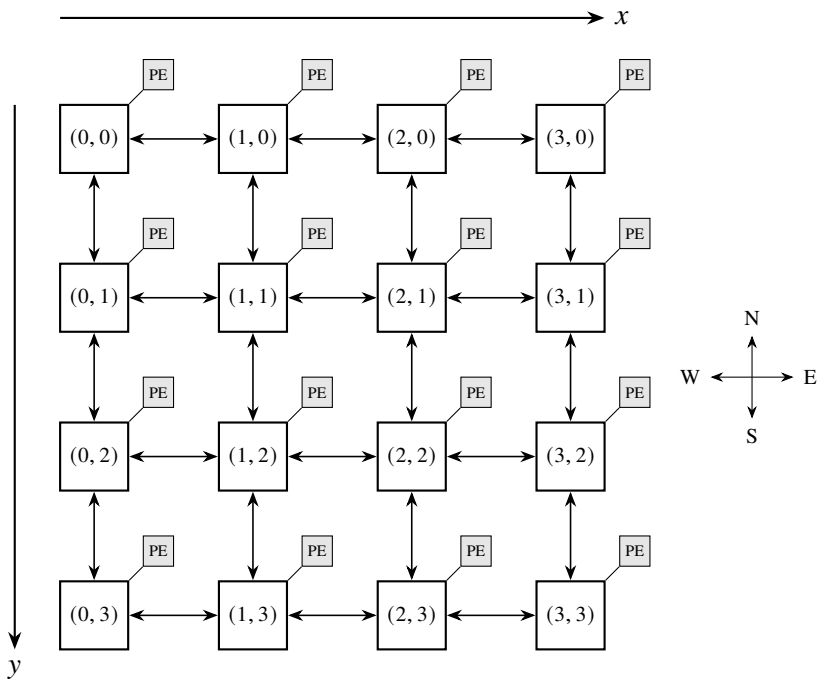


Figure 1: The 4×4 mesh. Each router is labelled with its address (x, y); each arrow is a pair of links, one in each direction. Every router also connects to one PE through its local port.

1.2 The router

Every router has five ports, each with an input and an output (Figure 2):

- four **network ports**, north, east, south and west, which carry packets to and from the neighbouring routers; and
- one **local port**, through which the attached PE puts packets into the network and gets the packets addressed to it.

The local port is the only part of the router's interface that is fixed: the router must provide it as a sub-interface of type `Ifc_noc_port` (Section 2). Everything else is yours to design: the interface between neighbouring routers, how much buffering there is and where it sits, how many cycles a packet spends in a router, how the output port is chosen, how contention for an output is resolved, and how a router that is full tells its neighbours to stop.

The router MUST be developed as an independent package `NocRouter`, in the file `NocRouter.bsv`. Ensure that the package is complete, and independently compiles. It must import only the `NocTypes` package and any other standard BSV library that you choose. Note that the NOC router should be reusable to construct a mesh of arbitrary size with minimal effort. The asssignment will not grade/test the NOC router on its own, but you are expected to submit a testbench that verifies basic functionality of the NoC router. This will carry points.

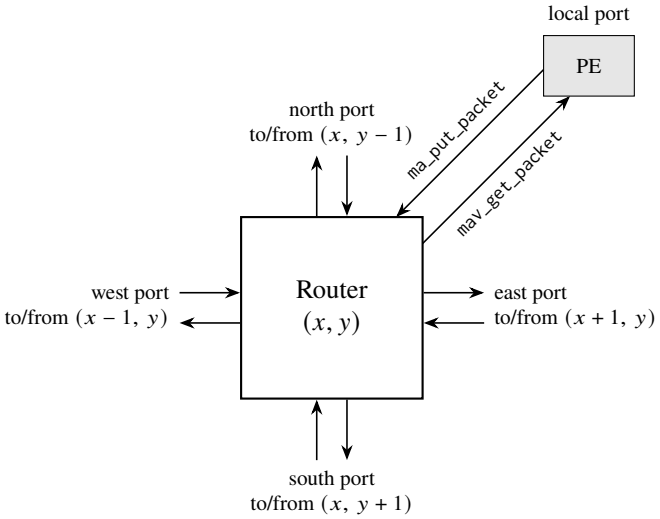


Figure 2: One router. Each of the five ports has an input and an output. On the edge of the mesh, the network ports that face off the grid are unused.

1.3 Packets and messages

Everything the network carries is a packet with the four fields of Table 1.

Field	Width	Meaning
`src`	2 + 2 bits	address (x, y) of the PE that sent the packet
`dst`	2 + 2 bits	address (x, y) of the PE the packet is for
`is_last`	1 bit	set on the last packet of a message, clear on every other packet
`seq_num`	3 bits	position of this packet within its message, counting from 0
`payload`	256 bits	32 bytes of data, opaque to the network
Total	268 bits	

Table 1: Packet format.

A PE communicates in *messages*. A message of up to 32 bytes travels as a single packet with `seq_num` = 0. A longer message is split into consecutive packets numbered 0, 1, 2, . . .; no message needs more than 8

packets, so `seq_num` never exceeds 7. The splitting is done by the PE — in this assignment, by our test bench — before the packets reach your router. The network only ever sees packets. The header does not carry the length of the message, but `is_last` marks its last packet.

Your network must deliver each packet exactly as it was put in, all 268 bits of it. Inside the network you may represent a packet however you like: you can attach fields of your own, such as a precomputed route or a tag, or move a packet across a link over several cycles in narrower pieces (*flits*), as long as what comes out of the destination's local port is identical to what went in at the source.

The widths above are parameters in `NocTypes.bsv`. We will test only the configuration of Table 1 and Figure 1, but write your design in terms of `MeshCols`, `MeshRows`, `CoordWidth`, `SeqWidth` and `PayloadBytes` rather than the numbers they currently hold, so that a different mesh size, sequence-number width or payload size is a one-line change. This is part of what we mean by a reusable router.

2 The supplied skeleton

You are given two files. `NocTypes.bsv` defines the parameters, the packet type and the two interfaces that the test bench uses:

```
package NocTypes;

import Vector :: *;

typedef 4   MeshCols;        // routers along x, left to right
typedef 4   MeshRows;        // routers along y, top to bottom
typedef 2   CoordWidth;      // bits per coordinate
typedef 3   SeqWidth;        // bits of sequence number
typedef 32  PayloadBytes;    // payload carried by one packet
typedef TMul#(PayloadBytes, 8) PayloadWidth;

typedef struct {
   Bit#(CoordWidth) x;
   Bit#(CoordWidth) y;
} NodeAddr deriving (Bits, Eq, FShow);

typedef struct {
   NodeAddr             src;
   NodeAddr             dst;
   Bool                 is_last;
   Bit#(SeqWidth)       seq_num;
   Bit#(PayloadWidth)   payload;
} NocPacket deriving (Bits, Eq, FShow);

// The port through which a PE talks to its router.
interface Ifc_noc_port;
   method Action ma_put_packet (NocPacket pkt);
   method ActionValue#(NocPacket) mav_get_packet ();
endinterface

// The whole network: node[x][y] is the local port of the router at (x, y).
interface Ifc_noc;
   interface Vector#(MeshCols, Vector#(MeshRows, Ifc_noc_port)) node;
endinterface

endpackage
```

The first index of `node` is x and the second is y, in the same order as the address, so the local port of the router at $(2, 0)$ is `node[2][0]`. In the generated Verilog, its signals are prefixed `node_x_y_`. Do not modify `NocTypes.bsv`: we compile your code against our own copy of it.

`Noc.bsv` contains the module you fill in:

```
package Noc;

import Vector    :: *;
import NocTypes  :: *;
import NocRouter :: *;

(* synthesize *)
module mkNoc (Ifc_noc);

   // Instantiate one router from your NocRouter package for every (x, y),
   // connect each router to its neighbours, and return each router's local
   // port as node[x][y].

endmodule

endpackage
```

`mkNoc` carries `(* synthesize *)`, and our test bench drives it both in Bluesim and as generated Verilog.

3 Functional specification

3.1 What the test bench guarantees

We understand that developing a full fledged and efficient NoC is a hard task, and are providing you with some guarantees to make it easier. We expect your design to hold up against these requirements and our testbench will never violate any of these. While you are free to explore and build the NoC further on your own, we will limit the scope to these requirements alone. **We however recommend that you think about these in detail as architectural and functional thinking is a very useful skill to pick up!**

A1. A packet put in at node (x, y) has `src` $= (x, y)$.

A2. `dst` is a node of the mesh, and never equal to `src`.

A3. Each PE puts in the packets of a message consecutively, in increasing order of `seq_num` from 0.

A4. Each PE has at most one message in the network at a time. It puts in the first packet of its next message only after every packet of the current message has been collected at the destination.

A5. Every packet that arrives at a node's local port is eventually collected, although a node may stop collecting for a long time.

A3 and A4 together mean that no two packets in the network at the same time share the same `src`, `dst` and `seq_num`. Those three fields identify a packet uniquely, so a reorder buffer can tell packets apart by them alone.

3.2 What your network must guarantee

G1. **Delivery.** Every packet accepted at a node's local port is returned exactly once, at the local port of the node named in its `dst` field, identical in all 268 bits to the packet that was put in. No packet may be lost, duplicated, corrupted or delivered to the wrong node.

G2. **Ordering.** For every pair of nodes S and D, the packets that S sends to D are returned at D in the order in which S put them in. Note that the source is guaranteed to put the packets in ascending order of sequence numbers, should there be any. This means you are expected to output the packets in the same order. **This is a hard requirement. While the packets themselves can get scrambled inside the NoC, your design must put them in order before it leaves the output port**.

G3. **Progress.** Whatever the traffic, every accepted packet is eventually delivered, and every PE that is

waiting to send is eventually allowed to. A network that deadlocks, or that holds a packet or a PE back indefinitely while others are served, fails.

3.3 Interface and timing contract

The local port follows the same discipline as the interface of Assignment 1, and the test bench relies on it.

Contract

C1. `ma_put_packet` may fire only when the router can genuinely accept the packet. Its `RDY` is your router's "full" signal towards the PE: when there is no room, the method is simply not ready, and the PE waits.

C2. `mav_get_packet` may fire only when a packet for that node is genuinely available. It must never return a stale, duplicated or partially assembled packet.

C3. Packets are returned in the order that G2 requires.

C4. Both conditions above are expressed as *implicit* conditions, so that the generated `RDY` signals carry them. Do not attach `(* always_ready *)` or `(* always_enabled *)` to either method, and do not build an unguarded handshake in which a separate method reports validity or fullness.

C1–C4 apply to the local port only. Between routers, you may use whatever handshake you like.

The test bench applies back-pressure on both sides. It will sometimes put packets in on consecutive cycles, as fast as `ma_put_packet` allows, and it will sometimes stop collecting at a node for many cycles. A packet whose destination is not collecting must wait inside the network, never be dropped, and back-pressure will then spread upstream; when the node starts collecting again, the network must drain.

3.4 Extreme load test

To make the assignment more challenging and to incentivize extensive thinking, there will be a specific amount of points alloted for performance under high load. For many thousands of cycles, all sixteen PEs will hammer the NoC with messages, as soon as the routers are ready to accept it. This will test if your design is robust to different kinds of workloads and if you honored the guarantees outlined above. A network that loses/duplicates/reorders/deadlocks/stalls under this load will not be awarded these points.

4 Performance

Alongside a working network, you must estimate how much bandwidth your router can support and how your network performs, and then measure both. We will measure the same quantities on your submission with our own traffic generators and compare them with yours. Table 2 defines them. There is no clock-period model in this assignment, so everything is counted in clock cycles, and bandwidth counts payload bytes only: header bits are overhead.

The bandwidth by area (B/W) fraction will have to be estimated after synthesis (using synthesis clock rate). This will be a measure of how efficient your design is.

For each quantity, first derive an estimate from your micro-architecture — B_R(as defined in Table 2), for instance, follows from how many packets your router can forward in a cycle and how often it can do so — then measure it with your own test bench, and explain any difference between the two. Measure B_R and B_N(as defined in Table 2) in steady state: discard a warm-up period at the start of a run, and count over a window long enough for the rate to settle.

Quantity	Unit	Definition
L_0, zero-load latency	cycles	From the cycle in which `ma_put_packet` fires at $(0,0)$ to the first cycle in which `mav_get_packet` is ready at $(3,3)$, for a single packet in an otherwise idle network.
B_R, router bandwidth	bytes/cycle	The highest rate at which one router can move payload from its inputs to its outputs in steady state, summed over all five outputs.
B_S, stream bandwidth	bytes/cycle	The rate at which payload arrives at $(3,3)$ while $(0,0)$ sends it one 8-packet message, with no other traffic and $(3,3)$ always collecting: 256 bytes divided by the number of cycles from the one in which the first packet is collected to the one in which the last is, inclusive.
B_N, network throughput	bytes/cycle	The total rate at which payload is delivered, summed over all sixteen nodes, when every node sends 8-packet messages back to back, each as soon as A4 allows, to destinations chosen uniformly at random from the other fifteen nodes.

Table 2: Performance quantities. Each must be both estimated and measured.

5 Design space exploration

Build at least three configurations of your design and compare them. **A configuration in this context means varying internal microarchitectural details of the design (pipeline depth, buffer size etc.) and not the dimensions of the NoC. You are only expected to deliver a 4x4 NoC.**

A configuration counts only if you take it through the whole flow:

(a) Compilation with Bluesim and Verilog flow.
(b) Passes your custom NoC test.
(c) All quantities in Table 2 have been measured for it.

Present the configurations in your report as a table giving, for each one, what changed, the four quantities of Table 2, and the total buffer storage in one router in bits, as a rough measure of its cost. You should submit the stats of your best design which will be considered for evaluation.

For the Physical Design

1. For all three designs explored, fill the following table after Synthesis. **Note Synthesis to be run for slow corner with 40% clock period for external delay.**

Design Feature	Closing Frequency with 0/+ve slack	Cell count and Design Area (from Synthesis qor)

Table 3: To be filled for Physical Design Evaluation

6 Constraints

1. Do not modify `NocTypes.bsv`.
2. In `Noc.bsv`, the package name, the module name, its interface and its `(* synthesize *)` attribute are fixed; the test bench will not compile against changed ones. You may add imports and helper functions.
3. `mkNoc` should do nothing but instantiate routers from your `NocRouter` package, one per node, and connect them to each other and to `node`. Anything each node needs, such as a reorder buffer, belongs in the router package.

4. `NocRouter.bsv` holds package `NocRouter`, imports only `NocTypes` and packages that ship with `bsc`, and compiles on its own. The names of the router module and its interface are your choice.
5. Size everything from the typedefs in `NocTypes.bsv`, not from the numbers they currently hold.
6. The design must be synthesisable, and `mkNoc` must compile through the Verilog flow.

7 Building and testing

`mkNoc` is a concrete module, so you can check that it compiles to Verilog on its own:

```
$ mkdir -p build
$ bsc -u -verilog -bdir build -vdir build -info-dir build -g mkNoc Noc.bsv
```

To simulate it, write a test bench that instantiates it and compile that for Bluesim:

```
$ bsc -u -sim -bdir build -simdir build -info-dir build -g mkTbNoc TbNoc.bsv
$ bsc -sim -bdir build -simdir build -e mkTbNoc -o simTbNoc
$ ./simTbNoc
```

You must write and submit two test benches:

1. a **router test bench**, which exercises one router on its own. It drives the router's ports directly and checks that each packet leaves by the output your routing policy chooses, that contention for an output loses nothing, and that back-pressure on an output stalls the right inputs; and
2. a **NoC test bench**, which drives `mkNoc` through `Ifc_noc` and checks G1–G3.

The NoC test bench must include, at the least, a single packet sent from $(0, 0)$ to $(3, 3)$ with nothing else in the network: the packet must arrive at $(3, 3)$, unchanged. You are free to explore further tests. Note that having high confidence on your NoC router will hugely reduce the possibility of having a faulty NoC itself.

The README.md file attached with the assignment package provides you with directions along with a basic starter testbench. This is so that you can understand how inputs would be driven in the evaluation tests. While you are free to modify that testbench to include your own tests, it is recommended that you adopt your own testbenches, while ensuring it adheres to the framework followed in the sample.

8 Evaluation

BSV Section :

Your submission is compiled against our `NocTypes.bsv` and our test bench, which instantiates `mkNoc` and drives all sixteen local ports with directed and randomised traffic under back-pressure, checking G1–G3 and C1–C4. The functional test cases range from a single packet in an idle network to many PEs sending at once; the extreme-load test of Section 3.4 is run separately. We also measure the quantities of Table 2 with our own traffic generators and compare them with the numbers in your report.

A submission that fails to compile scores zero, so leave time to check that the files you upload are the files you tested.

Component	Marks
Functional test cases, in proportion to the fraction passed	2
Correctness under extreme load (Section 3.4)	2
Standalone router package with its own test bench	1
Design space exploration: at least three configurations taken through the whole flow (Section 5)	1
Lab Quiz (PnR)	4
Total	**10**

Table 4: Mark distribution.

9 Deliverables

Submit a single archive named `<rollnumber>_assign2.zip` containing:

Please make sure you submit the correct files, We will be trying to replicate your numbers

1. `NocRouter.bsv` — your router package.
2. `Noc.bsv` — your completed `mkNoc`.
3. Your router test bench and your NoC test bench, with anything needed to run them.
4. `README` — how to build and run each test bench and each configuration. Please provide an overview of the different configs used in the exploration limited to a few lines each.
5. `bsv_results.pdf` - You will have to fill in a table of the template provided in the assignment. There must be one copy of this table for each of you design space explorations, and you must explicitly mention your best version which should be considered for evaluation. The table should contain populated values from Table 2. Ensure that you provide in (cycles or per cycles) and (nanoseconds or bytes/second).
6. `pd_results.pdf` - You will submit one pdf which will contain table 3 for each of the explored design.
7. `pd_reports` - you will submit 3 qor reports (corresponding to each Synthesis), renaming them to design_ < number>_qor.rpt.You will mention that the design number corresponds to what feature change in README.
8. `tcl and sdc files` - You will submit the tcl and sdc files used in each round of Synthesis after renaming them to design_<number>_<name>.tcl design_<number>_<name>.sdc

10 Use of AI tools

You may use AI tools as much as you like. We encourage educated and judicious use of those tools, under the condition that you own everything you submit. Owning something in this context means that you understand every line of your design and testbenches and be able to explain why it exists. You should also be able to explain your microarchitecture, and any decisions you made in the design. All numbers submitted as part of your reports should be real results and not fabricated.

Remember that the goal of these assignments, especially today, is to teach you to design and think, while also to work effectively with AI. **You should be proud of your work if at the end you feel that: you understand how NoCs work, how they are designed, the issues that could come up in a NoC, how to measure and quantify a NoC's performance and why they are generally better than a crossbar.**

11 Academic honesty

Discussing the specification and the language with your classmates is fine; sharing code, in either direction, is not, whether a person or an AI tool wrote it. Submissions are compared against each other and against public repositories, and a match is referred to the institute's disciplinary process without exception.

12 Contact

Questions about the specification should go to the course forum so that everyone sees the answer; anything that would reveal part of your solution should come by email instead.

Sanjeev Subrahmaniyan ee23b102@smail.iitm.ac.in
Abhinav I S ee23b002@smail.iitm.ac.in

For doubts regarding Physical Design, Docker Setup and Server Access feel free to email the following TAs

S K Hari Kheshav cs24s025@cse.iitm.ac.in
Jayanth R ns25z033@smail.iitm.ac.in

References

1. A reference for NoC designs: `https://liacs.leidenuniv.nl/~stefanovtp/courses/ES/papers/Ch15_NoC_Design.pdf`.